



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

«Информатика и системы управления»

КАФЕДРА

«Теоретическая информатика и компьютерные технологии»

Отчет о лабораторной работе № 2

по курсу «Операционные системы»

Загружаемый модуль ядра (драйвер)

Студент: Федукон А. А.

Группа: ИУ9-42Б

Преподаватель: Брагин А.В.

Москва, 2025 г.

Содержание

Постановка задачи	2
Часть 1. Драйвер в ReactOS	2
Часть 2. Драйвер в NetBSD	2
Практическая реализация	3
Часть 1. Драйвер в ReactOS	3
Часть 2. Драйвер в NetBSD	5
Результаты	7
Часть 1. Драйвер в ReactOS	8
Часть 2. Драйвер в NetBSD	8
Выводы	9
Список литературы	9

Постановка задачи

Цель работы — разработать простой загружаемый модуль ядра (драйвер), который выводит в отладочный журнал фамилию студента, выполнившего лабораторную работу. Драйвер должен компилироваться в отдельный файл, а не быть частью ядра.

Часть 1. Драйвер в ReactOS

В созданном в лабораторной работе № 1 рабочем дереве ОС ReactOS создать новый модуль, который представляет собой простейший драйвер, совместимый с ОС Windows NT и ReactOS. Драйвер должен реализовать минимальный набор функций, необходимый для загрузки и выгрузки этого драйвера. В функции инициализации этого драйвера DriverEntry осуществить вывод в отладочный лог, используя макрос DPRINT1, фамилии студента, выполнившего лабораторную работу.

Часть 2. Драйвер в NetBSD

В виртуальной машине с NetBSD, созданной в лабораторной работе № 1, создать новый загружаемый модуль ядра (loadable kernel module), реализующий простейший драйвер. Драйвер должен содержать минимальный набор функций, необходимый для загрузки и выгрузки этого драйвера. В функции инициализации драйвера осуществить вывод в отладочный лог фамилии студента, выполнившего лабораторную работу. Список загружаемых драйверов можно получить командой `find /stand -type f`, а затем найти код искомого модуля.

Практическая реализация

Часть 1. Драйвер в ReactOS

Для выполнения второй лабораторной работы я продолжил работу в уже существующем дереве исходного кода ReactOS, созданном в первой лабораторной. В этот раз я перешёл от использования QEMU из консоли к более удобной связке **KVM + Virt Manager**, которая позволяет гибко управлять виртуальными машинами через графический интерфейс. Для установки системы я использовал тот же ISO-образ ReactOS, который был собран ранее.

Создание структуры драйвера

В директории `reactos/drivers` я создал новую папку с названием `lab2`:

```
mkdir reactos/drivers/lab2
```

Затем я открыл файл `reactos/drivers/CMakeLists.txt` и в самый конец добавил строку:

```
add_subdirectory(lab2)
```

Это подключило мой модуль к системе сборки.

Добавление исходных файлов

В директории `lab2` я создал три файла, необходимых для сборки драйвера[3]:

- **lab2.c** — содержит реализацию минимального драйвера с функциями DriverEntry и DriverUnload, в которой с помощью макроса DPRINT1 выводится моя фамилия;
- **lab2.rc** — ресурсный файл с информацией о драйвере;
- **CMakeLists.txt** — файл сборочной системы CMake для компиляции драйвера.

Исходный код этих файлов представлен ниже:

Файл lab2.c

Листинг 1: Исходный код lab2.c

```

1 #include <ntddk.h>
2 #include <debug.h>
3
4 NTSTATUS
5 NTAPI
6 DriverEntry(
7     IN PDRIVER_OBJECT DriverObject,
8     IN PUNICODE_STRING RegPath)
9 {
10
11     DPRINT1("Lab2: Fedukov Alex");
12
13     return STATUS_SUCCESS;
14 }
```

Файл lab2.rc

Листинг 2: Исходный код lab2.rc

```

1 #define REACTOS_VERSION_DLL
2 #define REACTOS_STR_FILE_DESCRIPTION "Lab2Lib"
3 #define REACTOS_STR_INTERNAL_NAME "Lab2Lib"
4 #define REACTOS_STR_ORIGINAL_FILENAME "lab2lib.sys"
5 #include <reactos/version.rc>
```

Файл CMakeLists.txt

Листинг 3: CMake-файл для сборки драйвера

```
1 add_library(lab2 MODULE lab2.c lab2.rc)
2
3 set_module_type(lab2 kernelmodedriver)
4 add_importlibs(lab2 ntoskrnl)
5 add_cd_file(TARGET lab2 DESTINATION reactos/system32/drivers NO_CAB FOR
    all)
```

Сборка и подключение ISO

Далее я собрал только мой драйвер с помощью команды:

```
ninja lab2
```

После этого я создал новый ISO-образ, в который включил собранный драйвер, используя утилиту genisoimage:

```
genisoimage -o lab2.iso .
```

Затем подключил созданный ISO-образ к уже существующей виртуальной машине в Virt Manager, указав его как дополнительный CD-ROM.

Загрузка драйвера внутри ReactOS

После запуска системы я выполнил регистрацию и запуск драйвера вручную с помощью стандартных команд ReactOS (аналогично Windows):

```
sc create lab2driver binPath= C:\ReactOS\System32\drivers\lab2.sys type=kernel
sc start lab2driver
```

Проверка отладочного вывода

После запуска драйвера я проверил файл отладочного вывода (debug log), куда была успешно выведена моя фамилия с помощью макроса DPRINT1. Это подтвердило, что мой драйвер корректно загрузился и выполнил инициализацию.

Часть 2. Драйвер в NetBSD

В продолжение лабораторной работы я использовал уже установленную виртуальную машину с NetBSD, которую разворачивал ранее с использованием серийного вывода. В рамках этого этапа мне предстояло создать собственный загружаемый модуль ядра (loadable kernel module, LKM) — минимальный драйвер, который при инициализации выводит мою фамилию в отладочный лог.

Создание исходного файла драйвера

Сначала я создал файл `/usr/src/sys/dev/lab2.c` с минимальной реализацией драйвера. Код выглядит следующим образом:

Листинг 4: Исходный код `lab2.c` для NetBSD

```
1 #include <sys/module.h>
2 MODULE(MODULE_CLASS_MISC, lab2, NULL);
3 static int lab2_modcmd(modcmd_t cmd, void* arg) {
4     printf("driver LAB2");
5     return 0;
6 }
```

Создание Makefile для модуля

Далее я создал директорию `/usr/src/sys/modules/lab2/` и поместил туда Makefile, описывающий сборку моего модуля [3]:

Листинг 5: Makefile для модуля `lab2`

```
1 .include "../Makefile.inc"
2 KMOD=lab2
3 .PATH: ${S}/dev
4 SRCS=lab2.c
5 .include <bsd.kmodule.mk>
```

Сборка и загрузка модуля

Я перешёл в директорию `/usr/src/sys/modules/lab2` и выполнил сборку модуля командой:

```
make
```

В результате сборки был получен файл lab2.kmod, который представляет собой скомпилированный загружаемый модуль ядра.

Затем я выполнил загрузку модуля с помощью команды:

```
modload ./lab2.kmod
```

После выполнения этой команды моя фамилия отобразилась в логе, что подтвердило успешную инициализацию драйвера.

Проверка загруженных модулей

Для проверки наличия модуля в системе я использовал команду [4]:

```
modstat
```

А для чтения лога:

```
dmesg
```

Также я убедился, что модуль можно выгрузить при необходимости с помощью команды:

```
modunload lab2
```

Результаты

При запуски драйвера в лог выводилась моя фамилия. Таким образом, в результате выполнения лабораторной работы был создан и подключён в систему простой драйвер. Система корректно его загрузила и отобразила отладочную информацию, что подтверждает правильность сборки и понимание процесса взаимодействия драйверов с ядром системы.

```
Command Prompt
ReactOS [Version 0.4.16-x86-dev 20250329-unknown-revision]
(C) Copyright 1996-2025 ReactOS Team.

C:\Documents and Settings\Administrator>sc start lab2driver
SERVICE_NAME: lab2driver
        TYPE               : 1  KERNEL_DRIVER
        STATE                : 4  RUNNING
                               (STOPPABLE,NOT_PAUSABLE,IGNOR
        WIN32_EXIT_CODE       : 0  (0x0)
        SERVICE_EXIT_CODE   : 0  (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x0
        PID                 : 0
        FLAGS                 : 0

C:\Documents and Settings\Administrator>_

CFSFolder.cpp:1061: Unexpected failure (hr)=8000
CFSFolder.cpp:1061: Unexpected failure (hr)=8000
CDrivesFolder.cpp:744: Unexpected failure (hr)=8000
CRegFolder.cpp:506: Unexpected failure (hr)=8000
(win32ss/user/ntuser/window.c:2202) err: Thread
nnot create window (OleMainThreadWndClass)
(win32ss/user/ntuser/window.c:2796) err: co_User
err:(dll/win32/ole32/compobj.c:1721) CreateWindo
WARNING: SetThreadUILanguage at dll/win32/kerne
LEMENTED!
CFSFolder.cpp:1061: Unexpected failure (hr)=8000
CDrivesFolder.cpp:744: Unexpected failure (hr)=8000
CRegFolder.cpp:506: Unexpected failure (hr)=8000
err:(dll/win32/comctl32/edit.c:1248) es->text ==
(ntoskrnl/mm/ARM3/sysldr.c:170) Loading: \??\C:\V
s at F6716000 with 8 pages
(drivers/lab2/lab2.c:11) Lab2: Fedukov Alex
```

Рис. 1 — Вывод моей фамилии после запуска драйвера ReactOS

```
ssh root@192.168.122.54
hostname# pwd
/usr/src/sys/modules/lab2
hostname# ls
Makefile 1386 lab2.kmod.map machine xen
amd64 lab2.kmod lab2.o x86
hostname# modload ./lab2.kmod
hostname# dmesg | tail -5
[ 14.087926] wdisplay0: screen 3 added (80x25, vt100 emulation)
[ 14.087926] wdisplay0: screen 4 added (80x25, vt100 emulation)
[ 523.188313] WARNING: module error: module '--help' not found
[ 525.394249] WARNING: module error: module '-h' not found
[ 582.213759] driver LAB2 FEDUKOVdriver LAB2 FEDUKOVdriver LAB2 FEDUKOV
hostname#

2. Добавить минималистичную реализацию драйвера:
#include <sys/module.h>
MODULE(MODULE_CLASS_MISC, lab2, NULL);
static int lab2_modcmd(modcmd_t cmd, void* arg) {
    printf("driver LAB2");
```

Рис. 2 — Вывод моей фамилии после запуска драйвера NetBSD

Выводы

В ходе выполнения лабораторной работы я на практике освоил процесс создания простейших драйверов для двух различных операционных систем — ReactOS и NetBSD. В обоих случаях я реализовал минимальный загружаемый модуль, выводящий отладочную информацию при инициализации, что подтвердило успешную интеграцию моего кода в ядро системы.

Работа с ReactOS показалась мне более прямолинейной, однако здесь были нюансы с подготовкой ISO-образа и подключением его через виртуальную машину, что потребовало дополнительных знаний технологии виртуализации.

С NetBSD работа была более сложной технически, особенно из-за необходимости разбираться в структуре дерева исходного кода и механизмах сборки модулей. Зато документация оказалась более системной и подробной, что помогло разобраться в процессе.

В целом, я получил ценный опыт низкоуровневой работы с ядром операционной системы, научился собирать и подключать собственные модули, а также работать с отладочной информацией. Эти навыки помогут мне лучше понимать архитектуру ОС и принципы взаимодействия драйверов с ядром.

Список литературы

- [1] Официальная документация ReactOS
(<https://reactos.org/wiki/>)
- [2] Официальная документация NetBSD
(<https://www.netbsd.org/docs/>)
- [3] Учебно-методическое пособие курса "Операционные системы"
(<https://press.bmstu.ru/catalog/item/8226/>)
- [4] Рекомендованный tutorial по созданию netBSD драйверов
<http://www.home.unix-ag.org/bmeurer/NetBSD/howto-lkm.html>